

Pulse Studio Android 1.6.4 · Documentación técnica

Sep 29, 2026 · @Pablo Juzgado

Pulse Studio es una app Android (Java + WebView) que genera packs de TikTok a partir de una noticia: guion, seis imágenes verticales, miniatura con IA y vídeo editado con subtítulos. No tiene servidor propio: toda la lógica está en una interfaz web local y las claves de API son las del usuario.

1. Alcance y funciones

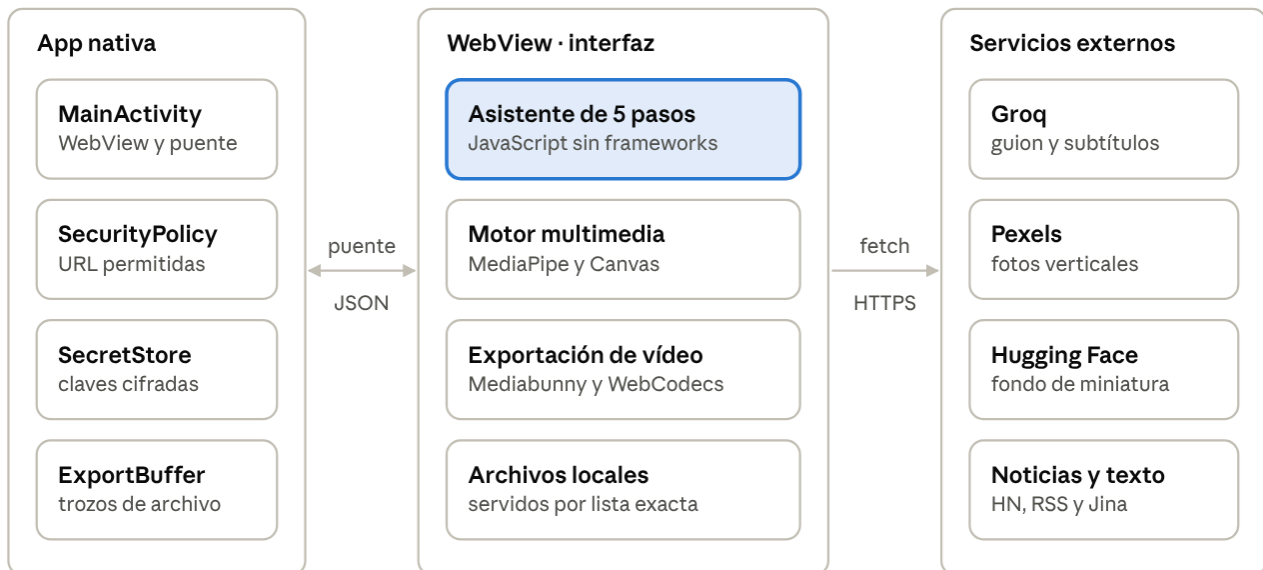
La app guía al usuario por cinco pasos. Cada paso es un panel del mismo asistente de pantalla única.

Paso	Función	Servicios que usa
1. Noticia	Buscar por tema y categoría, o introducir una noticia a mano	Hacker News (Algolia), RSS de HackRead, KrebsOnSecurity y Xataka
2. Generar	Leer el artículo, escribir el guion y buscar seis imágenes	Jina Reader, Groq, Pexels
3. Exportar	Copiar o guardar el guion (TXT) y las seis imágenes JPEG 1080 × 1920	Almacenamiento del sistema (SAF)
4. Miniatura	Portada 1080 × 1920 con la foto del usuario recortada, fondo de IA y titular	Groq, Hugging Face, MediaPipe (local)
5. Vídeo	Vídeo narrado sobre las seis imágenes, con recorte de fondo, subtítulos y exportación MP4	MediaPipe, Mediabunny (locales), Groq Whisper

Queda fuera de la app: cuentas de usuario, servidor propio, publicación directa en TikTok y cualquier almacenamiento en la nube. La foto y el vídeo del usuario no salen del dispositivo; a Groq solo llegan texto y la pista de audio para los subtítulos.

2. Arquitectura

La lógica vive en la WebView; Java solo guarda y protege



arquitectura de Pulse Studio Android · 3 zonas

La interfaz web (`index.html`) contiene toda la lógica de negocio y llama directamente a las APIs con las claves del usuario. La parte nativa solo aporta lo que una WebView no tiene: claves cifradas, guardado de archivos y una lista cerrada de URL. Se comunican por el canal `PulseNative` con mensajes JSON.

3. Stack, versiones y estructura

El código nativo es pequeño (unas 685 líneas de Java en cinco clases). La interfaz es un único `index.html` de unas 1 590 líneas, sin frameworks.

Componente	Versión
App	1.6.4 (versionCode 22)
Android mínimo	API 24 (Android 7.0)
compileSdk / targetSdk	36 / 36
Android Gradle Plugin / Gradle wrapper	8.13.2 / 8.14.3
Java	17
AndroidX WebKit / Core	1.16.0 / 1.17.0
MediaPipe Tasks Vision (local)	0.10.35

Componente	Versión
Mediabunny (local)	1.60.0
Android System WebView	97 o posterior (WebAssembly bajo CSP)

Ruta	Contenido
<code>app/src/main/java/com/pulsestudio/app/MainActivity.java</code>	Actividad única: WebView, puente nativo, guardado, selector de archivos
<code>SecurityPolicy.java</code>	Decisiones de URL: origen, página, activos locales, hosts de red y enlaces externos
<code>SecretStore.java</code>	Claves cifradas con AES-256-GCM y Android Keystore
<code>ExportBuffer.java</code> , <code>ImageBatchBuffer.java</code>	Recepción acotada y ordenada de archivos en fragmentos
<code>app/src/main/assets/www/index.html</code>	Interfaz completa (HTML, CSS y JavaScript)
<code>app/src/main/assets/www/vendor/</code>	MediaPipe (motor y modelo) y Mediabunny, servidos localmente
<code>app/src/main/res/xml/</code>	Configuración de red y reglas de exclusión de copias
<code>tests/</code>	Pruebas web (Node) y prueba de extremo a extremo con Chromium
<code>tools/</code>	<code>fetch-mediapipe.cjs</code> (descarga verificada del motor) y <code>update-csp.cjs</code> (hashes de la CSP)
<code>docs/</code>	Arquitectura, cambios y validaciones por versión

4. Puente nativo PulseNative

La interfaz solo puede pedir al sistema lo que la WebView no ofrece por sí sola: claves cifradas, guardado de archivos y avisos. Lo hace por un canal `WebMessageListener` llamado `PulseNative`, restringido al origen `https://appassets.androidplatform.net`.

Cada mensaje es un JSON con un `id` numérico (de 1 a 12 dígitos), una operación `op` y sus datos. Se descarta si pesa más de 70 000 caracteres, si no viene del marco principal o si la página no es la interna. La respuesta lleva el mismo `id`, `ok` y `result` o `error`. Las operaciones se ejecutan en un único hilo de E/S, en orden.

Operación	Función
<code>keys.load</code> / <code>keys.save</code> / <code>keys.clear</code>	Leer, guardar (Groq, Pexels y Hugging Face) o borrar las claves cifradas
<code>export.begin</code>	Abre una transferencia con nombre, tipo MIME, tamaño y destino. Devuelve un token
<code>export.chunk</code>	Envía un fragmento en Base64 (máximo 65 536 caracteres), con número de secuencia
<code>export.finish</code>	Cierra la transferencia. Destino <code>gallery</code> en Android 10+ guarda en MediaStore; si no, abre el selector de documentos
<code>export.abort</code>	Cancela y borra el archivo temporal
<code>images.begin</code> / <code>images.chunk</code> / <code>images.finish</code> / <code>images.abort</code>	Lo mismo para las seis imágenes del pack, con un selector de carpeta
<code>screen.keepOn</code>	Mantiene la pantalla encendida durante la exportación del vídeo
<code>external.open</code>	Abre un enlace <code>http</code> o <code>https</code> en el navegador (nunca uno interno)

Solo hay una transferencia a la vez. Los fragmentos deben llegar en orden y con tamaño acotado (48 KiB como máximo por fragmento en el búfer). Un token desconocido, un fragmento fuera de orden o un tamaño que no cuadra cancelan la operación. Los errores devuelven mensajes genéricos: nunca detalles del proveedor de cifrado ni de la ruta.

5. Seguridad

La WebView solo ejecuta código propio y solo habla con una lista cerrada de servidores.

Área	Medida
Contenido cargado	<code>WebViewAssetLoader</code> sirve <code>https://appassets.androidplatform.net/assets/www/index.html</code> y una lista exacta de archivos locales (<code>MediaPipe</code> y <code>Mediabunny</code>), con tipo MIME explícito. Nunca por prefijo de carpeta
Navegación	Cualquier página distinta de la interna se bloquea. Los enlaces externos se abren en el navegador solo si son <code>http</code> o <code>https</code> , sin usuario en la URL y con un gesto del usuario
Red	<code>shouldInterceptRequest</code> bloquea todo host que no esté en <code>SecurityPolicy: api.groq.com</code> , <code>api.pexels.com</code> , <code>images.pexels.com</code> , <code>r.jina.ai</code> , <code>hn.algolia.com</code> , <code>router.huggingface.co</code> , el worker RSS (solo <code>/feed</code>) y <code>huggingface.co</code> (solo <code>/api/models/*</code> y <code>/api/whoami-v2</code>). Solo HTTPS, sin puertos distintos de 443
Ajustes de WebView	Acceso a archivos y contenido desactivado, contenido mixto prohibido, cookies desactivadas, depuración web desactivada incluso en la versión de depuración, <code>SafeBrowsing</code> activo, permisos de cámara y micrófono denegados, errores SSL cancelados
CSP	<code>default-src 'none'</code> , <code>script-src 'self' 'wasm-unsafe-eval'</code> más tres hashes, <code>worker-src 'none'</code> , <code>object-src 'none'</code> . <code>'wasm-unsafe-eval'</code> permite WebAssembly sin habilitar <code>eval()</code>
Claves de API	AES-256-GCM con una clave no exportable del Android Keystore (alias <code>pulse.studio.credentials.v1</code>). Archivo <code>credentials.aesgcm</code> en <code>noBackupFilesDir</code> . Formato: 1 byte de versión, IV de 12 bytes y texto cifrado, con datos asociados <code>PulseStudio</code>
Permisos	Solo <code>INTERNET</code> . No hay <code>CAMERA</code> , <code>RECORD_AUDIO</code> , <code>READ_MEDIA_*</code> ni <code>WRITE_EXTERNAL_STORAGE</code> : los archivos se eligen con <code>ACTION_GET_CONTENT</code> y se guardan con <code>MediaStore</code> o <code>SAF</code>

Área	Medida
Copias de seguridad	<code>allowBackup=false</code> , <code>fullBackupContent=false</code> y reglas que excluyen todo de la copia en la nube y de la transferencia entre dispositivos
Tráfico en claro	Prohibido (<code>usesCleartextTraffic=false</code> y <code>network_security_config</code>)
Selector de archivos	Solo acepta <code>image/*</code> o <code>video/*</code> , un archivo, local y con esquema <code>content</code>
Proceso de la vista	Si Android cierra el proceso de la WebView, la app lo avisa y se cierra; las claves guardadas se conservan

La foto y el vídeo se procesan en el dispositivo. A Groq solo se envían el texto de la noticia y, para los subtítulos, la pista de audio; a Hugging Face, solo el prompt visual.

6. Módulos funcionales

6.1 Noticias y guion (pasos 1 a 3)

- **Búsqueda.** Combina Hacker News (API de Algolia) con tres fuentes RSS servidas por un worker propio (`/feed?source=`): HackRead, KrebsOnSecurity y Xataka. Filtra por tema con expresiones regulares, elimina duplicados por URL canónica y ordena por fecha.
- **Artículo.** Jina Reader devuelve el texto en Markdown. La app lo limpia (elimina menús, enlaces relacionados, pies de imagen y notas) y lo limita a unos 8 200 caracteres. El usuario puede pegar el texto a mano si el medio bloquea la extracción.
- **Guion.** Groq (`openai/gpt-oss-20b` por defecto, o `gpt-oss-120b`) responde con un esquema JSON estricto: gancho, contexto, desarrollo, por qué importa, cierre y seis palabras clave. El texto de la noticia va delimitado como datos, no como instrucciones.
- **Imágenes.** Pexels busca una foto vertical por palabra clave sin repetir. Cada una se recorta a 1080 × 1920 y se exporta como JPEG.

6.2 Miniatura (paso 4)

1. Groq propone un titular, una palabra destacada y un prompt visual en inglés (esquema estricto).
2. Hugging Face genera el fondo. La app consulta qué proveedor tiene el modelo en directo (`hf-inference` , `nscale` o `together`) y usa el primero. Modelos: FLUX.1 schnell, Stable Diffusion XL y Stable Diffusion 3 Medium. Sin token, el fondo puede ser

una de las seis imágenes del pack.

3. MediaPipe Selfie Segmenter recorta a la persona en el dispositivo. La foto se reduce a 1600 px como máximo. Si no detecta a nadie, la app prueba cuatro configuraciones en orden (GPU y CPU, con máscara de confianza y con máscara binaria) y conserva la que funciona.
4. Un canvas de 1080 × 1920 compone fondo, silueta con borde, persona y titular. No aparece el nombre del medio. Tamaño, posición y oscurecido se ajustan en tiempo real.
5. Se guarda como JPG o PNG en `Pictures/PulseStudio`.

6.3 Editor de vídeo (paso 5)

- **Entrada.** Vídeo narrado MP4 o WebM, de hasta 180 s y 500 MB.
- **Línea de tiempo.** Seis tramos, uno por imagen, con inicios ordenados y separados por un mínimo. Se pueden fijar con deslizadores, con «Aquí» (instante actual) o repartirlos por igual. Manteniendo pulsada una imagen de la barra y arrastrándola se reordena (pulsación larga de 320 ms). El orden solo afecta al vídeo.
- **Recorte de fondo.** MediaPipe segmenta cada fotograma a 320 px de lado largo, con suavizado temporal, y compone el sujeto sobre la imagen activa con fundido y zoom lento.
- **Subtítulos.** Mediabunny extrae la pista de audio sin recodificar. Groq `whisper-large-v3-turbo` la transcribe con tiempos por palabra (prompt: titular y guion). Las palabras se agrupan por número, pausa (más de 0,6 s) o puntuación. Ajustes: cinco tipos de letra, tres estilos (contorno, caja, sombra), color, mayúsculas, tamaño, posición vertical y palabras por subtítulo, más una lista para corregir los textos.
- **Exportación.** Fotograma a fotograma, sin depender de la velocidad del móvil: Mediabunny decodifica con WebCodecs a 30 fps exactos (aplicando la rotación), cada fotograma se compone y se codifica en H.264 (o HEVC, VP9 o AV1 si no hay H.264). El audio original se copia sin recodificar. El MP4 se escribe en trozos de 4 MB (`StreamTarget`, `fastStart: false`), cada uno como `Blob`, para no reservar nunca un bloque grande de memoria. Calidad de salida: 720 × 1280 o 1080 × 1920. Sin WebCodecs, se usa como respaldo la grabación en tiempo real con `MediaRecorder`.

7. Guardado de archivos y límites

La interfaz nunca elige rutas: manda bytes y un nombre validado, y Java decide dónde escribir.

Elemento	Destino	Límite
Guion (<code>.txt</code>)	Selector de documentos (SAF)	64 MiB

Elemento	Destino	Límite
Seis imágenes (<code>.jpg</code>)	Selector de carpeta (SAF). Crea <code>PulseStudio_<fecha></code> si el proveedor lo permite	64 MiB en total
Miniatura (<code>.jpg</code> , <code>.png</code>)	Android 10+: <code>Pictures/PulseStudio</code> (MediaStore). Antes: selector de documentos	64 MiB
Vídeo (<code>.mp4</code> , <code>.webm</code>)	Android 10+: <code>Movies/PulseStudio</code> (MediaStore). Antes: selector de documentos	256 MiB
Vídeo de entrada	Solo lectura, con <code>ACTION_GET_CONTENT</code>	500 MB y 180 s
Audio para subtítulos	Se envía a Groq	25 MB (límite del plan gratuito de Groq)

Los nombres deben cumplir `[a-zA-Z0-9_-]{1,100}` con extensión `txt` , `jpg` , `jpeg` , `png` , `mp4` o `webm` , y el tipo MIME debe coincidir con la extensión. Los archivos temporales viven en `cacheDir/exports` y se borran al cerrar, al cancelar y al arrancar. En MediaStore, el archivo se marca `IS_PENDING` mientras se copia y se elimina si la copia falla.

8. Rendimiento y memoria

La WebView corre en un proceso aparte. Si agota su memoria, Android lo cierra, así que las medidas están del lado de JavaScript.

Medida	Dónde
Exportación en trozos de 4 MB guardados como <code>Blob</code> , sin un bloque contiguo del tamaño del vídeo (corrige el error «Array buffer allocation failed» de la 1.6.4)	<code>createBlobWriter</code>
Inferencia de MediaPipe a 320 px y buffers de máscara reutilizados	<code>ReelRenderer.draw</code>
Un solo segmentador compartido, cerrado con <code>onTrimMemory</code> (nivel <code>UI_HIDDEN</code> o superior) o al ocultar la página	<code>PulseMedia.trimMemory</code>
<code>result.close()</code> de cada resultado de MediaPipe	<code>withMask</code>

Medida	Dónde
<code>ImageBitmap.close()</code> de foto y fondos; fondos del vídeo a 720 × 1280	<code>closeBitmap</code> , <code>ensureVideoBackgrounds</code>
<code>URL.revokeObjectURL()</code> de vídeos y vistas previas al cambiarlos o descartarlos	<code>releaseVideoSource</code> , <code>discardVideoResult</code> , <code>clearPack</code>
Canvas auxiliares a tamaño 0 al liberarlos	<code>freeCanvas</code>
Cancelar la exportación si la app pasa a segundo plano	<code>visibilitychange</code>
Pantalla encendida durante la exportación	<code>FLAG_KEEP_SCREEN_ON</code> (<code>screen.keepOn</code>)
Prioridad <code>IMPORTANT</code> del renderer y <code>largeHeap</code> en el proceso	<code>MainActivity</code> , manifiesto

Si la GPU no admite las texturas que MediaPipe necesita, el recorte cae automáticamente a CPU. El vídeo de salida siempre tiene 30 fps; en móviles lentos solo tarda más (con un proceso a 13 fps, un vídeo de 60 s tarda unos 2 min 20 s).

9. Compilación, instalación y pruebas

Compilar

1. Instalar Node 18 o superior y ejecutar una sola vez, desde la raíz del proyecto: `node tools/fetch-mediapipe.cjs`. Descarga `@mediapipe/tasks-vision@0.10.35` (verificado con SHA-512) y el modelo `selfie_segmenter.tflite` (SHA-256) en `assets/www/vendor/mediapipe/` (unos 19,5 MB). Sin ellos, Gradle no empaqueta la APK (tarea `checkMediaPipeAssets`).
2. Abrir la carpeta raíz con **Open** en Android Studio, instalar Android SDK Platform 36 y Build Tools 35.0.0, y usar JDK 17 o 21 como Gradle JDK.
3. Generar la APK de depuración y ejecutar las pruebas:

```
.\gradlew.bat assembleDebug testDebugUnitTest lintDebug
```

La APK queda en `app/build/outputs/apk/debug/app-debug.apk`. En macOS y Linux: `./gradlew` en lugar de `.\gradlew.bat`.

Si cambia un script de `index.html`, hay que regenerar los hashes de la CSP con `node tools/update-csp.cjs`.

Requisitos del dispositivo

Android 7.0 o superior y Android System WebView 97 o posterior (se actualiza desde Google Play). La exportación rápida necesita WebCodecs; sin ellos se usa el respaldo con `MediaRecorder`.

Pruebas

Capa	Herramienta	Qué cubre
Java (JVM)	JUnit 4.13.2, 12 pruebas	<code>SecurityPolicy</code> (origen, hosts, puertos, esquemas, activos locales), <code>ExportBuffer</code> (límites, secuencia, nombres, limpieza) e <code>ImageBatchBuffer</code>
Java (dispositivo)	AndroidX Test	<code>SecretStoreInstrumentedTest</code> : cifrado y Keystore. Se compila; hay que ejecutarla en un dispositivo
Web	Node con jsdom, 19 pruebas definidas	Extracción de artículos, CSP y flujo de la interfaz con datos simulados
Extremo a extremo	Playwright y Chromium	Miniatura (MediaPipe real), reordenación de imágenes, subtítulos, exportación MP4 de 30 fps. Los servicios externos se simulan
Análisis estático	Android Lint	<code>abortOnError true</code>

La evidencia de la versión 1.4.0 está en `docs/VERIFICATION.md` y `docs/evidence/`.

Consta un pendiente de verificación manual en dispositivos reales: SAF con proveedores en la nube, rotación, TalkBack, cuotas reales de Groq y Pexels, y firma de release.

10. Historial de versiones

Versión	versionCode	Cambio principal
1.6.4	22	Exportación del MP4 en trozos de 4 MB (<code>StreamTarget</code>): corrige «Array buffer allocation failed» en móviles reales. Lo mismo para el audio de los subtítulos

Versión	versionCode	Cambio principal
1.6.3	21	Reordenar las seis imágenes con pulsación larga y arrastre. Subtítulos automáticos con Groq Whisper y ajustes de estilo
1.6.2	20	Exportación de vídeo fotograma a fotograma con WebCodecs y Mediabunny: MP4 a 30 fps exactos. Operación nativa <code>screen.keepOn</code>
1.6.1	19	Recorte de persona con escalera de cuatro configuraciones (GPU y CPU, máscara de confianza y binaria). Sin nombre del medio en la miniatura
1.6.0	18	Paso 4 (miniatura con IA) y paso 5 (editor de vídeo). Hugging Face en Configuración
1.5.0	sin dato	Rediseño del flujo en pasos y de la pantalla de configuración. Selector de carpeta único para las seis imágenes. Se elimina la exportación ZIP

11. Limitaciones conocidas y licencias

Limitaciones

- **Créditos de Hugging Face.** El plan gratuito da pocos créditos al mes. Al agotarse (HTTP 402), la app lo indica y permite usar una imagen del pack como fondo.
- **Tiempo de exportación.** En móviles de gama baja, con recorte a 1080p, puede tardar dos o tres veces la duración del vídeo.
- **Formato de salida.** Depende del WebView del dispositivo: normalmente MP4 H.264; si no, WebM.
- **Recorte de persona.** Está pensado para personas en primer plano. Con fondos parecidos a la ropa o al pelo, el borde puede ser imperfecto.
- **Artículos.** Jina Reader no garantiza acceso a todos los medios (muros de pago o bloqueos). El usuario puede pegar el texto.
- **Contenido generado.** El guion, el titular y el fondo de IA deben revisarse antes de publicar.
- **Alcance de las pruebas.** Las automáticas cubren JVM, jsdom y Chromium con servicios simulados. El uso con claves reales se probó a mano en el emulador de Android Studio y en un móvil físico; no hay pruebas automáticas en dispositivo.

Licencias de terceros

Componente	Licencia
MediaPipe Tasks Vision 0.10.35 y modelo Selfie Segmenter (Google)	Apache 2.0
Mediabunny 1.60.0 (Vanilagy)	MPL 2.0 (bundle sin modificar, con su LICENSE)
AndroidX Core y WebKit, Gradle wrapper	Apache 2.0
JUnit, AndroidX Test, jsdom, Playwright	Solo pruebas; no van en la APK

Los servicios externos (Groq, Pexels, Hugging Face y los proveedores que enruta, Jina Reader, Hacker News y las fuentes RSS) tienen sus propias condiciones de uso. Las fotos de Pexels mantienen su atribución. El proyecto no incluye claves de API ni fotografías de terceros.

Fuentes

Todos los datos salen del proyecto `PulseStudioAndroid-v1.6.4`:

- `app/build.gradle` (versiones, SDK y dependencias) y `app/src/main/AndroidManifest.xml` (permisos y ajustes de la aplicación).
- `MainActivity.java`, `SecurityPolicy.java`, `SecretStore.java`, `ExportBuffer.java` e `ImageBatchBuffer.java` (puente, seguridad, cifrado y límites).
- `res/xml/network_security_config.xml` y `res/xml/data_extraction_rules.xml`.
- `app/src/main/assets/www/index.html` (límites de vídeo, fuentes RSS y operaciones del puente).
- `README.md`, `THIRD_PARTY_NOTICES.md`, `docs/ARQUITECTURA_v1.6.0.md`, `docs/CHANGELOG_v1.6.0.md` y `docs/VERIFICATION.md`.